Hanoi University of Science and Technology

School of Information and Communication Technology



WEB SEMANTICS

PROJECT REPORT

CineGraph: Linked Data for Films

A Semantic Movie Catalog and Knowledge Graph

Team members	**Student ID**
Bui Tien Dung	20261204M
Pham Tuan Kiet	20252751M
Luong Minh Duong	20251038M

Instructor: PhD. Do Ba Lam

October 7, 2026

Contents

1 Introduction **1**
1.1 Ontology Definition 1

2 Data Collection and Transformation **5**
2.1 Data Collection 5
2.2 Data Transformation 6
2.3 Audit Crawled Data 7

3 Linking Data to External Datasets **9**

4 Publishing SPARQL endpoint **11**

5 Conclusion **13**

References **14**

1. Introduction

Film information is distributed across catalogs, CSV datasets, APIs, and general-purpose knowledge bases. These sources describe overlapping films but use different formats and have different authority over individual fields. Titles alone are unreliable identifiers because remakes, localized names, and spelling variants can refer to distinct works. CineGraph integrates movie records by IMDb ID and represents the reconciled information as a knowledge graph.

The project builds a semantic movie catalog from IMDb, Kaggle, and OMDb, enriches local entities with external links to Wikidata and DBpedia, transforms the integrated records into RDF, validates the resulting graph with SHACL, and exposes it through a SPARQL endpoint. The supplied snapshot contains 3,651 retained movies from 4,671 IMDb candidates with more than 50,000 votes. RDF provides the underlying triple representation[6].

The catalog is a popularity-filtered, multi-source snapshot rather than a representative sample of all cinema. External knowledge bases are used to connect local entities to the wider Semantic Web; they do not add new movies to the retained catalog.

1.1 Ontology Definition

The ontology in `ontology/cinegraph.ttl` defines the vocabulary and class/property relationships. The normalized JSON catalog is the intermediate integration dataset. The saved `data/rdf/movies.ttl` contains both a copy of the ontology and instance assertions. Together those assertions form the knowledge graph; the RDF file is not an ontology-only artifact.

1.1.1 Classes and Role Modeling

The seven declared classes are Movie, Person, Actor, Director, Genre, ExternalRating, and FieldProvenance. Movie is a subclass of `schema:Movie`; Person is a subclass of `schema:Person`. Actor and Director specialize Person. They are not declared disjoint: a single person URI can have both roles. Direct inspection finds 178 people who are both actors and directors in the retained catalog. Using one identity preserves their shared film relationships instead of creating separate actor and director copies.

Actor and Director here describe credited roles, not time-qualified occupations. The model does not represent when a person held a role, individual cast ordering, or a separate performance event. Genre is a resource so films can join through a shared node. ExternalRating separates its supplying service from its original score string; FieldProvenance separates an integrated field's origin from the movie's overall source membership. Awards remain a text summary because the active importer does not resolve individual award events.

1.1.2 Property Specification and Constraints

Table 1.1 summarizes the actual declarations. Object properties connect resources; datatype properties attach literals. Cardinality checks are implemented separately through SHACL, not through OWL cardinality axioms in this ontology.

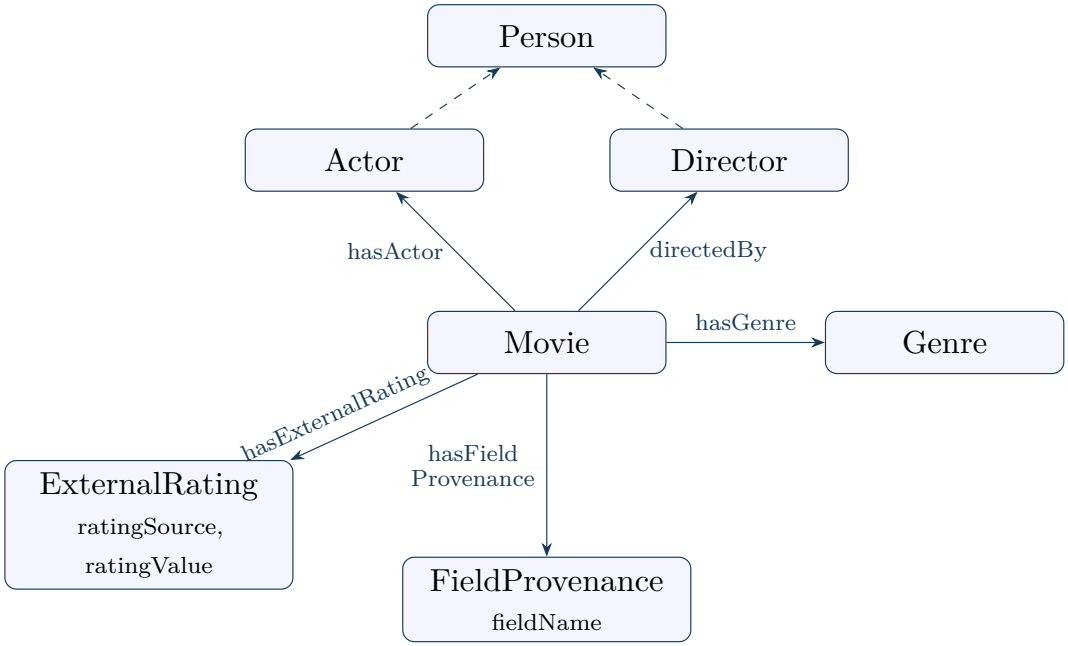


Figure 1.1. Ontology structure with rating values and field provenance. Dashed arrows denote subclass relationships. Source IRIs are shown in the provenance example.

Table 1.1. Selected domain, range, and property-type declarations.

Property	**Type**	**Domain**	**Range**
`cine:hasActor`	Object	Movie	Actor
`cine:directedBy`	Object	Movie	Director
`cine:hasGenre`	Object	Movie	Genre
`cine:hasExternalRating`	Object	Movie	ExternalRating
`cine:hasFieldProvenance`	Object	Movie	FieldProvenance
`cine:title`, `cine:description`	Datatype	Movie	`rdf:langString`
`cine:imdbID`	Datatype	Movie	`xsd:string`
`cine:releaseYear`	Datatype	Movie	`xsd:gYear`
`cine:imdbRating`	Datatype	Movie	`xsd:decimal`
`cine:durationMinutes`	Datatype	Movie	`xsd:decimal`
`cine:ratingVotes`	Datatype	Movie	`xsd:integer`
`cine:fieldName`	Datatype	FieldProvenance	`xsd:string`

1.1.2.1 Validation Constraints

Domain and range axioms imply types; SHACL checks required fields[5]. The web application runs no general OWL reasoner, and pySHACL is invoked with inference disabled. Explicit instance typing therefore supports the tested class constraints.

Table 1.2. Validation constraints, distinct from ontology axioms.

Field/relation	Implemented SHACL requirement
Title	Exactly one literal.
IMDb ID	Exactly one value matching `^tt[0-9]+$`.
Release year	At most one `xsd:gYear` value.
IMDb rating	At most one decimal value in $[0, 10]$.
Duration	At most one nonnegative decimal value.
Rating votes	At most one nonnegative integer value.
Actor/director/genre relations	IRI values with the corresponding declared entity class; no minimum count.
Person/genre labels	At least one readable literal label.

The production IMDb-ID shape now enforces the identity invariant with both `sh:minCount 1` and `sh:maxCount 1`. The ontology ranges of `cine:title` and `cine:description` are now `rdf:langString`, matching the builder's English-tagged literals. These declarations are consistent with the saved graph, and the final graph passes the production SHACL shapes with zero violations.

1.1.3 Vocabulary Reuse and Namespaces

The local vocabulary fixes the names used by application queries, field provenance, and dataset-specific metrics. It also duplicates some concepts already expressible in Schema.org. The design therefore favors a stable local query vocabulary over exclusive reuse. Table 1.3 states the actual alignment mechanism rather than assuming equivalent properties that are not declared.

Table 1.3. Actual reuse and alignment, without undeclared equivalence axioms.

Concept	Reuse mechanism in the implementation
Movie, Person	`rdfs:subClassOf` Schema.org classes; explicit instance types.
Names and descriptions	Local literals and parallel `schema:name`/`schema:description`.
Cast and directors	Local edges and parallel `schema:actor`/`schema:director` edges.
Genre	Local resource link plus a literal `schema:genre` label.
Provenance	`prov:wasDerivedFrom` and `dcterms:source`; local field node.
External identity	`owl:sameAs`; distinct from links to human-readable pages.

1.1.3.1 URI Design Strategy

Under https://example.org/cinegraph/, deterministic paths use `movie/IMDb-ID`, `person/tmdb-ID`, and `genre/slug`. Rating and provenance paths include the movie ID and source/field name. People without TMDB IDs use name slugs, which can collide after normalization. These inspectable identifiers are stable under repeated ingestion, but name slugs are not globally unique. Public dereferencing requires a controlled namespace.

Table 1.4. Namespaces used in RDF construction and semantic queries.

Prefix	Namespace
`cine:`	`https://example.org/cinegraph/ontology#`
`cg:`	`https://example.org/cinegraph/`
`schema:`	`https://schema.org/`
`rdf:`	`http://www.w3.org/1999/02/22-rdf-syntax-ns#`
`rdfs:`	`http://www.w3.org/2000/01/rdf-schema#`
`owl:`	`http://www.w3.org/2002/07/owl#`
`prov:`	`http://www.w3.org/ns/prov#`
`dcterms:`	`http://purl.org/dc/terms/`
`xsd:`	`http://www.w3.org/2001/XMLSchema#`

2. Data Collection and Transformation

2.1 Data Collection

Separate adapters handle raw downloads, IMDb parsing, Kaggle CSV ingestion, OMDb enrichment, external links, and person profiles. Table 2.1 summarizes their roles. Reconciliation operates at field level because format compatibility alone does not determine which source should supply a value.

Table 2.1. Source roles in the active pipeline.

Source	Input and identity	Contribution
IMDb	Compressed basics and ratings TSV joined on `tconst`.	Title, year, runtime, genres, IMDb rating and votes.
Kaggle	Metadata and credits CSV; credits joined by TMDB movie ID, movies by IMDb ID.	Revenue, budget, cast, directors, TMDB IDs and rating fields.
OMDb	JSON requested with exact IMDb ID.	Plot, poster, awards and external rating pairs.
Wikidata	SPARQL/API; IMDb IDs for movies, TMDB IDs for people.	Identity links and linked-person profiles.
DBpedia	SPARQL matched by IMDb ID.	External movie identity links.

2.1.1 IMDb Candidate Selection

The adapter downloads or reuses `title.basics.tsv.gz` and `title.ratings.tsv.gz`. Official dataset documentation describes their identifiers and columns[2]. Ratings are indexed by `tconst`; only records with `numVotes>50000` enter the default candidate set. The basics file is filtered to `titleType=movie` and joined to the ratings. The comparison is exclusive: exactly 50,000 votes does not pass. The threshold can be changed through `CINEGRAPH_MIN_IMDB_VOTES`.

The stored snapshot reports 4,671 selected candidates. This is a popularity-filtered subset, not the size of IMDb. Shared parsing helpers handle missing or malformed numeric fields instead of treating them as valid zero-valued measurements.

2.1.2 Kaggle Metadata and Credits

The configured dataset is The Movies Dataset, identified in source by `rounakbanik/the-movies-dataset`. The downloader requests `movies_metadata.csv` and `credits.csv`. Credits are joined to metadata through TMDB movie ID. Nested cast and crew arrays are parsed; up to eight cast entries are retained per input movie, Director crew entries are extracted, and numeric TMDB person IDs are preserved when available.

The pipeline summary records 45,466 imported Kaggle records and 3,651 IDs matching the selected IMDb candidates. Its 41,766 unmatched IMDb IDs are a distinct-identifier count, whereas imported records count CSV rows. Duplicates and missing IDs mean these quantities should not be compared by simple subtraction. Enrichment records without an

exact IMDb match are skipped.

2.1.3 OMDb Requests and Cache

OMDb is queried only for records already matched between IMDb and Kaggle. Requests specify the IMDb ID and a short plot. Responses are cached per identifier under `data/cache/omdb/`; reusable cached responses do not consume the budget for new requests. The adapter handles invalid cache files, request failures, and quota errors.

The API key is read through `OMDB_API_KEY`. Default settings allow 700 new requests per run; the stored run records a configured budget of 1,000 and zero new requests. It contains 3,651 usable OMDb results and zero pending targets. These are cache-backed snapshot observations, not API throughput measurements. Without an API key, the active pipeline cannot produce the intended three-source catalog.

2.1.4 Catalog Membership

Let I be the vote-filtered IMDb movie ID set, K the compatible Kaggle ID set, and O the IDs with usable OMDb records. The final catalog is

$$C = I \cap K \cap O. \tag{2.1}$$

The pipeline retains only records whose source flags include all three sources. It keeps 3,651 of 4,671 candidates and excludes 1,020, giving a retention rate of 78.16%. Kaggle records and external linked entities never add extra movies to this catalog.

2.2 Data Transformation

The transformation pipeline reconciles records by stable identifiers, merges source-specific fields, builds RDF resources and relationships, and serializes the resulting knowledge graph as Turtle. The active workflow keeps only movies represented by all three primary movie sources before RDF construction.

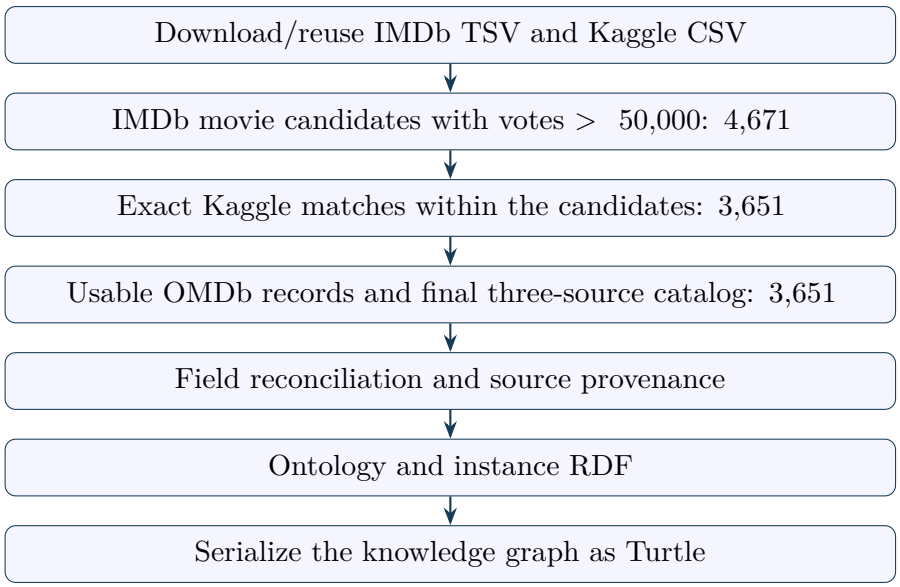


Figure 2.1. Data integration and RDF transformation pipeline.

2.2.1 Field Reconciliation

The merge function compares source priorities per field. A higher-priority nonempty value replaces the current value; an empty field can be filled by a lower-priority source; ties keep the earlier value. IMDb takes precedence for title, year, duration, genres, rating and votes. Kaggle supplies budget, revenue, and normally cast and director credits. OMDb takes precedence for plot, poster and awards.

The source-membership map identifies contributing datasets. A separate field-source map records which source supplied the final value for fields governed by the reconciliation rules. This allows the integrated record to preserve its source lineage rather than treating all merged values as if they came from the same dataset.

2.2.2 RDF Construction and Provenance

The RDF builder emits typed movies, reusable people and genre resources, literal attributes, external ratings, identity links, and provenance. Movie-level `prov:wasDerivedFrom` and `dcterms:source` assertions identify contributing sources. FieldProvenance nodes associate a local field name with its supplier using PROV-O[4].

```
@prefix cine: <https://example.org/cinegraph/ontology#> .
@prefix prov: <http://www.w3.org/ns/prov#> .

<https://example.org/cinegraph/movie/tt1375666> cine:hasFieldProvenance <https://example.org/cinegraph/
     provenance/tt1375666/plot>,
        <https://example.org/cinegraph/provenance/tt1375666/revenue>,
        <https://example.org/cinegraph/provenance/tt1375666/title> ;
    cine:imdbID "tt1375666" ;
    cine:title "Inception"@en .

<https://example.org/cinegraph/provenance/tt1375666/plot> a cine:FieldProvenance ;
    prov:wasDerivedFrom <https://www.omdbapi.com/> ;
    cine:fieldName "plot" .

<https://example.org/cinegraph/provenance/tt1375666/revenue> a cine:FieldProvenance ;
    prov:wasDerivedFrom <https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset> ;
    cine:fieldName "revenue" .

<https://example.org/cinegraph/provenance/tt1375666/title> a cine:FieldProvenance ;
    prov:wasDerivedFrom <https://www.imdb.com/interfaces/> ;
    cine:fieldName "title" .
```

Listing 2.1. Field provenance for Inception: title, revenue, and plot.

For example, Inception's title originates in IMDb, revenue in Kaggle, and plot in OMDb. External ratings retain their original source/value pairs rather than being averaged across incompatible scales. Movie years are represented as `xsd:gYear` literals, while person data may preserve different date precision depending on the external source.

2.3 Audit Crawled Data

The final knowledge graph is checked using SHACL before publication. SHACL expresses machine-readable constraints over RDF nodes and properties, allowing the project to verify required values, datatypes, cardinalities, numeric ranges, and relation targets[3]. Validation is executed with inference disabled so that the reported result reflects explicitly materialized graph statements.

The main checks require every movie to have exactly one title and one IMDb identifier,

constrain the IMDb identifier to the pattern `^tt[0-9]+$`, require valid datatypes for year, rating, duration and vote count, and check that actor, director and genre links point to resources of the expected classes. Numeric constraints additionally restrict IMDb ratings to $[0, 10]$ and require nonnegative duration and vote counts.

Table 2.2. SHACL checks applied to the final knowledge graph.

Field/relation	**SHACL requirement**
Title	Exactly one literal value.
IMDb ID	Exactly one value matching `^tt[0-9]+$`.
Release year	At most one `xsd:gYear` value.
IMDb rating	At most one `xsd:decimal` value in $[0, 10]$.
Duration	At most one nonnegative `xsd:decimal` value.
Rating votes	At most one nonnegative `xsd:integer` value.
Actor/director/genre	IRI values that point to the corresponding entity class.
Person/genre labels	At least one readable label.

The complete saved Turtle graph contains 645,850 triples. It was parsed and validated against the production SHACL shapes, and the final graph conforms with **zero SHACL violations**. This result shows that the published snapshot satisfies the structural and datatype constraints defined for the project; it does not claim correctness beyond the constraints encoded in the SHACL shapes.

3. Linking Data to External Datasets

External linking connects CineGraph resources to established entities on the Semantic Web. Because `owl:sameAs` states that two URIs identify the same individual, the linker relies on stable identifiers rather than title or name similarity alone[5]. Human-readable pages are represented separately with links such as `foaf:page`.

Movie entities are resolved against Wikidata using IMDb identifier property `wdt:P345` and against DBpedia using `dbo:imdbId`. Person entities are resolved against Wikidata using TMDB person identifiers from movie credits. Genre entities use English labels together with film-genre restrictions, because no equivalent stable identifier is available in the local source data.

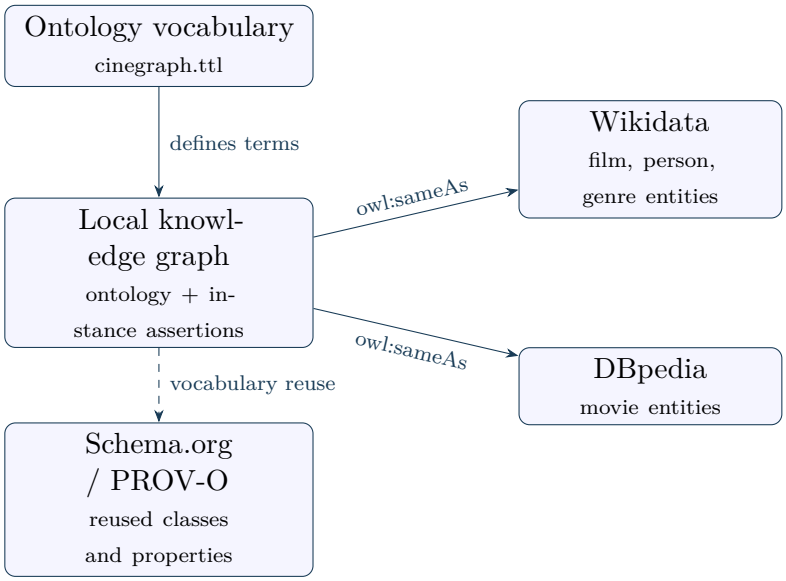


Figure 3.1. Local RDF resources and their connections to external datasets.

The saved snapshot contains Wikidata links for all 3,651 retained movies and DBpedia links for 87 movies. Among 13,504 people with TMDB identifiers, 12,336 are linked to Wikidata. Nineteen of the 22 supported genre labels are also linked to Wikidata. Person enrichment retrieves available descriptions, dates, images and Wikipedia pages for already linked people; it does not change the membership of the movie catalog.

Table 3.1. External-link coverage in the saved snapshot.

Metric	Count	Coverage
Movies linked to Wikidata	3,651 / 3,651	100.00%
Movies linked to DBpedia	87 / 3,651	2.38%
TMDB-identified people linked to Wikidata	12,336 / 13,504	91.35%
Supported genres linked to Wikidata	19 / 22	86.36%
Linked people with profile data	12,295 / 12,336	99.67%
Linked people with images	8,528 / 12,336	69.13%

The graph uses external identity links to improve interoperability, but the demonstrated

deployment remains local. A public Linked Open Data release would additionally require a controlled persistent namespace, dereferenceable resource URIs, stable hosting, and appropriate redistribution permissions for the contributing datasets.

4. Publishing SPARQL endpoint

The saved Turtle graph is loaded with RDFLib and exposed through a Flask `/sparql` route. GET or POST requests provide a SPARQL SELECT query through the `query` field, and successful responses are returned as SPARQL Results JSON. The endpoint supports the graph patterns, filters, optional patterns, ordering and aggregation needed by the movie catalog[1].

The endpoint is intentionally read-only: update operations and SERVICE clauses are rejected. The saved graph contains 645,850 triples. At application startup, a small set of derived person-count and property-description triples is added for the local interface, producing 659,358 runtime triples. Free-form SELECT queries operate on the full loaded graph.

Typical queries can retrieve films by genre, director, rating, cast membership, or external identity. Listing 4.1 shows a query that retrieves movies in the Sci-Fi genre, while Listing 4.2 retrieves the retained movies directed by Christopher Nolan.

```
PREFIX cine: <https://example.org/cinegraph/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?title ?genre WHERE {
  ?movie a cine:Movie ; cine:title ?title ; cine:hasGenre ?genreNode .
  ?genreNode rdfs:label ?genre .
  FILTER(LCASE(STR(?genre)) IN ("sci-fi", "science fiction"))
}
ORDER BY ?title
```

Listing 4.1. Example SPARQL query: movies by genre.

```
PREFIX cine: <https://example.org/cinegraph/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?title ?year WHERE {
  ?movie a cine:Movie ; cine:title ?title ; cine:directedBy ?director .
  OPTIONAL { ?movie cine:releaseYear ?year }
  ?director rdfs:label "Christopher Nolan"@en .
}
ORDER BY ?year
```

Listing 4.2. Example SPARQL query: movies by director.

The web interface demonstrates the same semantic graph through searchable movie and person views, preset SPARQL questions, and a bounded neighborhood visualization. Related movies are explained using shared genre, actor and director resources rather than an opaque recommendation score. The current endpoint is hosted locally at http://127.0.0.1:5000; therefore it demonstrates publication and querying of the RDF graph without claiming a persistent public SPARQL service.

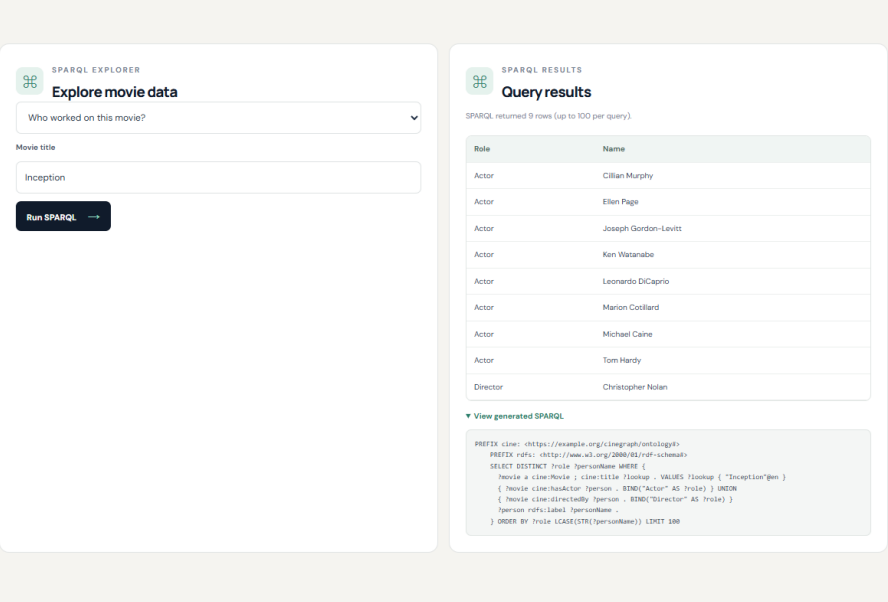


Figure 4.1. SPARQL exploration in the CineGraph application.

5. Conclusion

CineGraph demonstrates an end-to-end Semantic Web workflow for movie data. Records from IMDb, Kaggle, and OMDb are reconciled through stable IMDb identifiers, transformed into an RDF knowledge graph, enriched with external links to Wikidata and DBpedia, and exposed through a local SPARQL endpoint. The retained snapshot contains 3,651 movies and 645,850 saved triples.

The ontology models movies, people, actor and director roles, genres, external ratings, and field-level provenance. SHACL is used as the data-quality validation layer: the complete saved graph passes the production SHACL constraints with zero violations. This confirms conformance to the structural, cardinality and datatype rules explicitly defined for the project.

The current implementation remains limited by the popularity-based IMDb filter, the requirement that movies occur across the selected sources, bounded cast extraction, incomplete external-link coverage for some datasets, and local rather than public deployment. Future work can expand source coverage, improve entity resolution, publish persistent dereferenceable URIs, and deploy the SPARQL endpoint as a stable public service.

References

[1] Steve Harris and Andy Seaborne. SPARQL 1.1 Query Language. https://www.w3.org/TR/sparql11-query/, 2013. W3C Recommendation, 21 March 2013.

[2] IMDb. IMDb Non-Commercial Datasets. https://data.imdb.com/non-commercial-datasets/. Dataset documentation, accessed 7 October 2026.

[3] Holger Knublauch and Dimitris Kontokostas. Shapes Constraint Language (SHACL). https://www.w3.org/TR/shacl/, 2017. W3C Recommendation, 20 July 2017.

[4] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. PROV-O: The PROV Ontology. https://www.w3.org/TR/prov-o/, 2013. W3C Recommendation, 30 April 2013.

[5] W3C OWL Working Group. OWL 2 Web Ontology Language Primer (Second Edition). https://www.w3.org/TR/owl2-primer/, 2012. W3C Recommendation, 11 December 2012.

[6] W3C RDF Working Group. RDF 1.1 Concepts and Abstract Syntax. https://www.w3.org/TR/rdf11-concepts/, 2014. W3C Recommendation, 25 February 2014.